

Asynchronous Layer-Wise Weight Updating for Communication-Efficient Federated Learning in IoT Health Devices

Thomas Llamzon*, Mostafa Fouda[†], Mohamed I. Ibrahim[‡], Sherief Hashima[§], and Zubair Md Fadlullah*

*Department of Computer Science, Western University, London, ON, Canada.

[†]Department of Electrical and Computer Engineering, Idaho State University, Pocatello, ID, USA.

[‡]Department of Cyber Systems Engineering, Augusta University, Augusta, GA, USA.

[§]Computational Learning Theory Team, RIKEN-AIP, Fukuoka, Japan.

Emails: tllamzon@uwo.ca, mfouda@ieee.org, mibrahem@augusta.edu, Sherief.hashima@riken.jp, zfadlullah@ieee.org

Abstract—Federated Learning (FL) enables collaborative model training across distributed Internet of Things (IoT) health devices without centralizing sensitive patient data. However, standard synchronous FL suffers from high communication overhead, as clients upload the entire model parameters in each round, severely constraining bandwidth-limited IoT devices. This paper proposes a periodic asynchronous layer-wise update schedule that partitions model parameters into shallow and deep groups, transmitting shallow parameters every round and synchronizing deep parameters every K rounds. We evaluate this approach for ECG classification on the PTB-XL dataset using a 1D Convolutional Neural Network across centralized, synchronous, and asynchronous FL architectures. In a controlled 3-client, 4-round simulation, asynchronous FL with $K = 2$ reduces client upload volume by 49.70% and total communication by 24.85% with no measurable cost in accuracy. Across three random seeds under IID partitioning, asynchronous $K = 2$ reaches 83.18% mean accuracy (standard deviation 0.76) against a synchronous FedAvg baseline of 82.68% (standard deviation 0.98), a mean paired difference of 0.50 percentage points that is not statistically distinguishable from zero ($p = 0.31$). Under non-IID partitioning with a Dirichlet concentration of 0.5, a single run reaches 84.62% against a synchronous baseline of 84.53%. At $K = 4$, upload reduction reaches 74.55% in exchange for a clear accuracy loss (79.21% under IID, single run). These results demonstrate that periodic layer-wise scheduling can substantially reduce uplink communication cost in resource-constrained FL deployments while preserving acceptable model utility.

Index Terms—Federated Learning, IoT, ECG Classification, Communication Efficiency, Asynchronous Updates

I. INTRODUCTION

Federated Learning (FL) enables collaborative model training across distributed Internet of Things (IoT) devices while preserving data privacy, making it suitable for healthcare applications such as wearable electrocardiogram (ECG) monitoring. In these settings, devices generate sensitive physiological signals that cannot be centralized due to privacy and regulatory constraints. FL addresses data locality by exchanging model parameters instead of raw data, but it introduces a major bottleneck, communication cost. Fig. 1 illustrates the privacy-

preserving learning setting, where raw ECG data remain local and only scheduled model updates are communicated.

Standard FL, particularly Federated Averaging (FedAvg), follows a synchronous round-based protocol where clients download the global model, perform local training, and upload the complete updated model to the server. For deep ECG classifiers with millions of parameters, repeated full-model uploads dominate the cost of training. This is especially problematic for IoT health devices, where uplink bandwidth, energy, and connectivity are constrained.

Communication inefficiency also affects energy consumption and latency in IoT health deployments, where frequent uplink transmissions draw on battery-powered devices and can increase training delay. This motivates communication-aware learning strategies that reduce unnecessary transmissions without sacrificing performance; we address the transmitted volume directly and treat the downstream energy and latency effects as motivation rather than as measured outcomes.

Two properties of deep networks make uniform synchronization wasteful. First, parameter mass is highly uneven: in our ECG classifier the two convolutional blocks nearest the input account for 0.60% of all parameters, so the uplink cost is almost entirely attributable to the remaining layers. Second, networks exhibit layer-wise convergence heterogeneity, with shallow layers stabilizing early while deeper layers continue adapting, which means the deep group is also the group best able to tolerate delayed aggregation. Together these suggest synchronizing the small, foundational shallow group at every round and deferring the large, staleness-tolerant deep group.

A key inefficiency is that conventional FL treats all model layers uniformly as depicted in Fig. 2. However, deep networks learn hierarchically. Shallow layers capture general low-level signal features, while deeper layers encode task-specific representations and classification boundaries. This motivates a layer-aware communication strategy where different parts of the model are synchronized at different temporal frequencies.

In this paper we study the simplest realizable form of layer-wise update scheduling: a fixed partition of the model into shallow and deep parameter groups, with shallow layers synchronized every round and deep layers every K rounds. Unlike

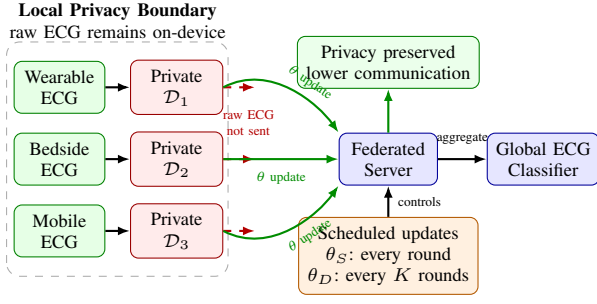


Fig. 1. Privacy-preserving federated learning for IoT health monitoring, where raw ECG records remain on local devices and only scheduled model updates are transmitted to the server.

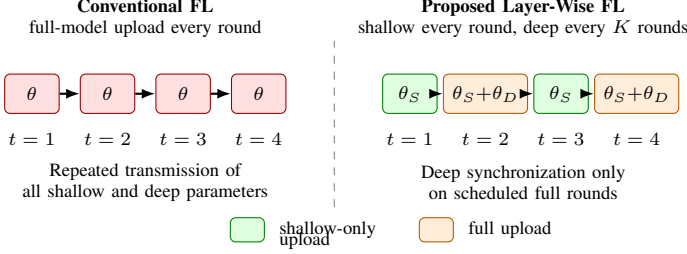


Fig. 2. Conceptual comparison between conventional full-model FL and the proposed periodic layer-wise update strategy.

adaptive layer-wise schemes, the partition and the interval are set in advance, so the server maintains no per-layer statistics and the method reduces to a scheduling rule over unmodified FedAvg. We characterize the resulting communication and accuracy behaviour on PTB-XL ECG classification under IID and non-IID partitions. At $K = 2$ upload cost falls by 49.70% with accuracy statistically indistinguishable from synchronous FL over three seeds, and at $K = 4$ upload reduction reaches 74.55% with a clear accuracy cost.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the considered system model and problem formulation. Section IV describes the proposed methodology and algorithm. Section V provides complexity analysis. Section VI presents performance evaluation. Section VII concludes the paper.

II. RELATED WORK

Federated Learning (FL) was popularized by McMahan et al. [1] through the Federated Averaging (FedAvg) algorithm, where a central server coordinates training by aggregating locally updated client models. Subsequent work by Yang et al. [2] formalized the FL paradigm into horizontal, vertical, and transfer settings. Healthcare IoT applications, such as ECG monitoring, fall under horizontal FL, where devices share the same feature space but operate on different patient data distributions.

A major limitation of FL is communication overhead. Konečný et al. [3] showed that communication rounds dominate overall training time, particularly in heterogeneous environments with varying device and network capabilities. To address this, prior work has focused on reducing the

size of transmitted updates. Konečný et al. [4] introduced structured and sketched updates, including sparse parameter selection, probabilistic quantization, and low-rank factorization, demonstrating substantial reductions in uplink volume; these techniques are surveyed in [5]. While compression-based approaches [6] and optimization-aware methods [7], [8] have been extensively studied for reducing communication overheads, the aforementioned approaches apply uniform compression across all model layers and do not exploit the hierarchical structure of deep networks.

An alternative direction is to modify the computation architecture. Yuan et al. [9] proposed a split learning approach for healthcare IoT, where shallow layers are executed on-device and deeper layers are offloaded to a central server. Combined with sparsification of exchanged activations and gradients, this method achieves substantial communication reduction while maintaining accuracy. However, it changes the standard FL architecture and retains synchronous round assumptions, without adapting communication frequency across different parts of the model.

Another line of work focuses on relaxing synchronization constraints. Asynchronous FL methods [10] allow clients to update the server without strict round barriers, improving system efficiency under heterogeneous conditions. While effective in reducing idle time, these approaches typically treat the model as a single parameter block and do not differentiate between layers in terms of communication frequency. Recent surveys [11] further highlight that communication efficiency remains a central challenge in IoT and edge FL deployments, especially under non-IID data and unreliable networks. Recent work has explored layer-wise adaptive federated learning [12], but without incorporating temporal scheduling of parameter updates within standard FL rounds.

Layer-wise scheduling itself is not new. FedLAMA [12] adjusts the aggregation interval in a layer-wise manner, jointly considering model discrepancy and communication cost, and maintains per-layer statistics at the server to do so. Despite these advances, layer-wise scheduling has so far been posed as a runtime adaptation problem, in which per-layer synchronization intervals are derived from measured model discrepancy and maintained by the server. We ask a narrower question: how much of the benefit survives when the schedule is fixed in advance and requires no measurement, no coordination, and no per-layer state at all. Our contribution is therefore not the idea of layer-wise scheduling but a characterization of its simplest realizable form, a static two-group partition chosen by parameter mass with a single constant interval, evaluated in a resource-constrained IoT health setting.

III. CONSIDERED SYSTEM MODEL AND PROBLEM FORMULATION

We consider an IoT healthcare FL system with one coordinating server and N distributed ECG clients. Each client i stores a private local dataset $\mathcal{D}_i = \{(\mathbf{x}_{i,j}, y_{i,j})\}_{j=1}^{n_i}$, where $\mathbf{x}_{i,j} \in \mathbb{R}^{5000 \times 12}$ denotes a 12-lead ECG segment and $y_{i,j} \in$

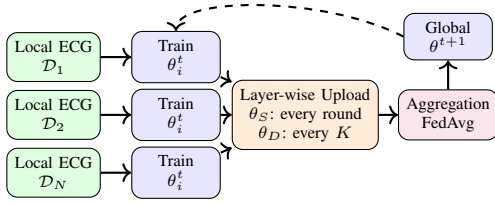


Fig. 3. Considered system model for asynchronous layer-wise FL.

$\{0, 1\}$ denotes the binary label, normal or abnormal. Raw ECG records remain local and are never transmitted to the server.

Let θ denote the full model parameters. The conventional FL objective is

$$\min_{\theta} F(\theta) = \sum_{i=1}^N \frac{n_i}{n} F_i(\theta), \quad F_i(\theta) = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell(f_{\theta}(\mathbf{x}_{i,j}), y_{i,j}), \quad (1)$$

where $n = \sum_i n_i$, F_i is the empirical local loss, and $\ell(\cdot)$ is the binary cross-entropy loss. The global objective captures collaborative learning across heterogeneous clients, where each local distribution may differ due to patient-specific characteristics, acquisition devices, or noise conditions. In practical IoT healthcare deployments, such statistical heterogeneity leads to non-IID data distributions [13], [14], which are known to slow convergence and destabilize federated optimization. Therefore, any communication-efficient strategy must maintain robustness under both IID and non-IID conditions.

In standard synchronous FL, each participating client uploads the full parameter vector θ_i^t at every round. The communication cost therefore scales with the full model size. In contrast, we partition the model into shallow and deep components,

$$\theta = (\theta_S, \theta_D), \quad (2)$$

where θ_S contains early convolutional layers and θ_D contains later convolutional and fully connected layers. The goal is to reduce uplink communication while preserving model utility. This gives the following communication-aware design problem:

$$\min_{\{\theta^t\}_{t=1}^T} F(\theta^T) \quad \text{s.t.} \quad \sum_{t=1}^T C_{\text{up}}(t) \leq B, \quad (3)$$

where B is the available uplink budget. This formulation follows standard federated optimization frameworks [1], [13]. The formulation in (3) implicitly assumes that all model parameters contribute equally to the communication cost, which is not the case in deep neural networks. In our ECG classifier the two convolutional blocks nearest the input hold 0.60% of the parameters, while the remaining layers hold 99.40%. Uplink cost is therefore almost entirely determined by the deep group, and synchronizing the shallow group is nearly free. This asymmetry, not convergence speed, is what makes a differentiated schedule worthwhile: sending the inexpensive group at every round and the expensive group periodically is the only partition that yields a meaningful reduction in uplink volume.

Convergence heterogeneity enters as a separate question, namely which group can tolerate being stale. Prior work on feature transferability [15] shows that early layers learn general representations while deeper layers specialize for the target task. Deep layers encode slower-changing, task-specific structure and therefore tolerate delayed aggregation, whereas the shallow layers form the shared feature extractor that every deep layer builds upon. Keeping that extractor consistent across clients at every round is what keeps the periodically aggregated deep layers mutually coherent. The two arguments are complementary: parameter mass determines which group is expensive to send, and staleness tolerance determines which group can afford to wait. Rather than solving this constrained problem directly, we adopt a periodic layer-wise schedule that controls when deep parameters are transmitted. The above formulation highlights the trade-off between model accuracy and communication cost. Reducing communication by limiting parameter transmission may introduce staleness in the global model, particularly for deep parameters that are not updated every round. The design challenge is therefore to determine a scheduling strategy that minimizes communication while preserving sufficient update frequency for critical model components.

IV. PROPOSED METHODOLOGY

We use the term *asynchronous* in a layer-wise rather than a client-wise sense, and the distinction matters. Conventional asynchronous FL removes the round barrier between clients, allowing each client to update the server independently and at its own pace. Our scheme retains the standard synchronous round structure and full client participation; what varies asynchronously is *which parameter groups are synchronized in a given round*. Shallow parameters are aggregated every round, deep parameters every K rounds, so different parts of the model are updated on different schedules and the deep group carries a bounded staleness of at most $K - 1$ rounds. The method is therefore complementary to client-wise asynchrony rather than an instance of it, and the two could be combined.

This section describes the experimental framework and the proposed asynchronous layer-wise scheduling strategy. The goal is to isolate the impact of temporal layer-wise communication on learning performance and communication cost.

A. Experimental Design

We compare three training architectures, centralized training, synchronous FL, and asynchronous layer-wise FL. The centralized setup provides a non-federated reference point, synchronous FL serves as the FedAvg baseline, and the asynchronous variant applies the proposed periodic scheduling. All settings use the same dataset, preprocessing, model architecture, optimizer, and training budget.

All runs fix the Python, NumPy, and PyTorch random seeds and pin the PyTorch intra-op thread count; a single training step was verified bitwise identical at 1, 2, 3, 4, 8, and 10 threads, so results are invariant to host CPU load and core count. For a given seed the synchronous and asynchronous

runs consume identical partitions and have bitwise identical round-zero checkpoints, so the schedules are compared from the same initialization on the same data. The IID synchronous and $K = 2$ configurations are repeated at three seeds; the rest are single runs.

B. Dataset and Model

Experiments use the PTB-XL dataset [16], a 12-lead ECG benchmark with 21,799 records sampled at 500 Hz. Each input is a 10-second ECG signal of shape (5000, 12). The task is binary classification between Normal and Abnormal rhythms. Folds 1–9 are used for training and fold 10 is used for testing.

The ECGCNN model contains 37,130,657 parameters, approximately 148.5 MB. The shallow group consists of conv1, bn1, conv2, and bn2, totaling 897,936 bytes, or 0.60% of the model. The deep group consists of conv3, bn3, conv4, bn4, fc1, fc2, and fc3, totaling 147,628,564 bytes, or 99.40%. This highly imbalanced partition is useful for testing whether infrequent deep synchronization can reduce communication without collapsing accuracy.

C. Federated Setup

The federated setting uses three simulated clients. Training runs for four communication rounds, with one participating client per round and one local epoch per selected client. Local training uses Adam with learning rate 10^{-3} and batch size 32. Both IID and non-IID partitions are evaluated. The non-IID split uses a Dirichlet distribution with $\alpha = 0.5$.

D. Asynchronous Layer-Wise Schedule

At round t , let $\mathcal{P}_t$ be the participating client set. The server computes sample-weighted aggregation for shallow and deep parameters as

$$\text{Agg}_S^t = \sum_{i \in \mathcal{P}_t} w_i^t \theta_{S,i}^t, \quad \text{Agg}_D^t = \sum_{i \in \mathcal{P}_t} w_i^t \theta_{D,i}^t, \quad (4)$$

where $w_i^t = n_i / \sum_{j \in \mathcal{P}_t} n_j$.

The shallow group is updated every round:

$$\theta_S^{t+1} = \text{Agg}_S^t. \quad (5)$$

The deep group is updated periodically:

$$\theta_D^{t+1} = \begin{cases} \text{Agg}_D^t, & t \bmod K = 0, \\ \hat{\theta}_D^t, & \text{otherwise,} \end{cases} \quad (6)$$

where $\hat{\theta}_D^t$ is the cached deep parameter state from the most recent full update.

The uplink cost at round t is

$$C_{\text{up}}(t) = |\mathcal{P}_t| (|\theta_S| + |\theta_D| \mathbb{I}[t \bmod K = 0]), \quad (7)$$

while the downlink cost is

$$C_{\text{down}}(t) = |\mathcal{P}_t| (|\theta_S| + |\theta_D|). \quad (8)$$

The proposed scheduling mechanism introduces a trade-off between communication efficiency and parameter freshness.

Algorithm 1 Periodic Asynchronous Layer-Wise FL

```

1: Initialize global model  $\theta^0 = (\theta_S^0, \theta_D^0)$ 
2: Initialize cached deep parameters  $\hat{\theta}_D^0 \leftarrow \theta_D^0$ 
3: for  $t = 1, \dots, T$  do
4:   Server selects participating clients  $\mathcal{P}_t$ 
5:   Server broadcasts full model  $\theta^t$ 
6:   for each client  $i \in \mathcal{P}_t$  do
7:     Client trains locally for one epoch to obtain  $\theta_i^t$ 
8:     if  $t \bmod K = 0$  then
9:       Client uploads  $(\theta_{S,i}^t, \theta_{D,i}^t)$ 
10:    else
11:      Client uploads  $\theta_{S,i}^t$  only
12:    end if
13:  end for
14:  Aggregate shallow parameters:
15:   $\theta_S^{t+1} \leftarrow \sum_{i \in \mathcal{P}_t} w_i^t \theta_{S,i}^t$ 
16:  if  $t \bmod K = 0$  then
17:    Aggregate deep parameters:
18:     $\theta_D^{t+1} \leftarrow \sum_{i \in \mathcal{P}_t} w_i^t \theta_{D,i}^t$ 
19:    Update cache  $\hat{\theta}_D^{t+1} \leftarrow \theta_D^{t+1}$ 
20:  else
21:    Reuse cached deep parameters:
22:     $\theta_D^{t+1} \leftarrow \hat{\theta}_D^t$ 
23:  end if
24:  Form updated global model  $\theta^{t+1} = (\theta_S^{t+1}, \theta_D^{t+1})$ 
25: end for

```

While shallow layers are continuously updated, deep parameters may become stale between full synchronization rounds. However, since shallow layers stabilize feature extraction, they can maintain consistent intermediate representations even when deep parameters are temporarily outdated. This results in a form of partial consistency, where feature quality is preserved while classification layers are intermittently refined. The effectiveness of this mechanism depends on the scheduling period K and the relative contribution of each parameter group to the overall model.

V. COMMUNICATION AND COMPLEXITY ANALYSIS

Let $P_S = |\theta_S|$ and $P_D = |\theta_D|$ denote the number of transmitted bytes for shallow and deep parameters, respectively. Let $M_t = |\mathcal{P}_t|$ be the number of participating clients at round t . In synchronous FL, each client uploads the full model every round, giving

$$C_{\text{sync}}^{\text{up}} = \sum_{t=1}^T M_t (P_S + P_D). \quad (9)$$

For the proposed asynchronous schedule, deep parameters are uploaded only on full rounds. Thus,

$$C_{\text{async}}^{\text{up}} = \sum_{t=1}^T M_t P_S + \sum_{t=1}^T M_t P_D \mathbb{I}[t \bmod K = 0]. \quad (10)$$

If $M_t = M$ for all rounds and K divides T , this simplifies to

$$C_{\text{async}}^{\text{up}} = M T P_S + M \frac{T}{K} P_D. \quad (11)$$

The corresponding uplink reduction relative to synchronous FL is

$$\rho_{\text{up}} = 1 - \frac{P_S + \frac{1}{K}P_D}{P_S + P_D}. \quad (12)$$

From an optimization perspective, the periodic omission of deep parameter updates can be interpreted as a form of delayed gradient aggregation. While such delays may slow convergence, their impact is mitigated when the delayed components correspond to parameters with slower update dynamics. In the considered ECGCNN architecture, deep layers evolve more gradually compared to shallow layers, making them more tolerant to reduced update frequency. This partially explains why moderate scheduling periods, such as $K = 2$, maintain accuracy while achieving significant communication savings.

Since $P_D \gg P_S$ in the ECGCNN model, the reduction is approximately $1 - 1/K$. Therefore, $K = 2$ yields close to 50% upload reduction, while $K = 4$ yields close to 75% upload reduction. This matches the empirical reductions of 49.70% and 74.55%, respectively.

The computational complexity of local training is unchanged because each client trains the full model after receiving the global model. If one local epoch costs $\mathcal{O}(n_i P)$ for client i , where $P = P_S + P_D$, then both synchronous and asynchronous FL have the same local training order. The main reduction is therefore not in floating-point computation but in uplink transmission. On the server side, aggregation cost is reduced during shallow-only rounds from $\mathcal{O}(MP)$ to $\mathcal{O}(MP_S)$, while full rounds retain $\mathcal{O}(MP)$ complexity. Thus, the proposed method reduces both communication load and partial server aggregation cost without changing client-side model execution.

From a system perspective, the reduction in uplink volume suggests several potential benefits that we did not measure. Lower peak uplink demand during shallow-only rounds may reduce contention in multi-client deployments, and fewer transmitted bytes may reduce the energy spent on radio activity in battery-powered devices. We note these as expected consequences rather than findings. Our evaluation measures transmitted bytes only; we did not measure device power draw, wall-clock latency, packet loss, or behaviour under unstable network conditions, and byte savings alone do not straightforwardly predict energy savings, since radio startup and association overhead can dominate the per-transmission energy cost.

VI. PERFORMANCE EVALUATION

This section evaluates the proposed asynchronous layer-wise FL scheme in terms of accuracy and communication cost. We compare centralized training, synchronous FL, and the proposed method under both IID and non-IID data distributions. All experiments use identical model architecture, training budget, and hyperparameters to isolate the effect of the scheduling mechanism.

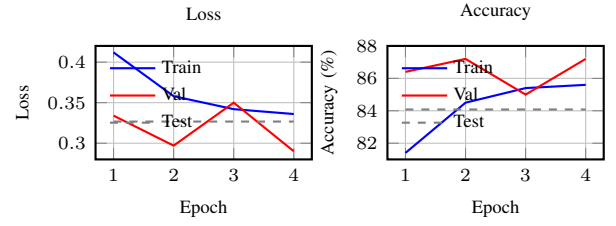


Fig. 4. Centralized training curves.

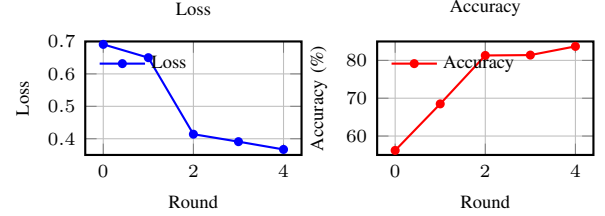


Fig. 5. Synchronous FL under IID partitioning.

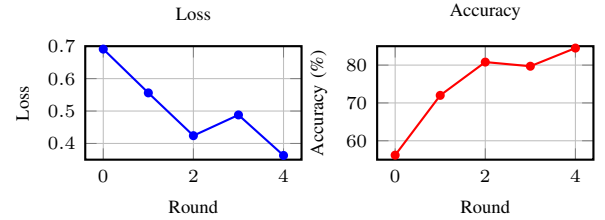


Fig. 6. Synchronous FL under non-IID partitioning.

A. Centralized and Synchronous Baselines

The centralized CNN achieves 84.08% test accuracy with loss 0.3267 after four epochs, establishing a non-federated reference point. We note that some federated configurations exceed this value, so it is a reference rather than a bound. Fig. 4 shows stable convergence, with training and validation curves indicating that the model effectively captures the ECG classification task.

Under IID partitioning, synchronous FL achieves $82.68 \pm 0.98\%$ accuracy over three seeds (81.85%, 82.44%, 83.76%); the run shown in Fig. 5 reaches 81.85% at loss 0.4015. Under non-IID partitioning with Dirichlet $\alpha = 0.5$, a single run achieves 84.53% accuracy with loss 0.3634, as shown in Fig. 6. The comparable performance under non-IID in this setting is plausibly due to the small number of clients and moderate heterogeneity, but with a single run per partitioning scheme we cannot separate that explanation from seed variation, which alone spans 1.91 percentage points under IID.

In both cases, full-model synchronization results in approximately 3.4 GB total communication, highlighting the inefficiency of transmitting all parameters at every round.

B. Asynchronous FL Results

The case $K = 1$ reduces to standard synchronous FL and achieves comparable performance, validating the correctness of the implementation, as shown in Fig. 7.

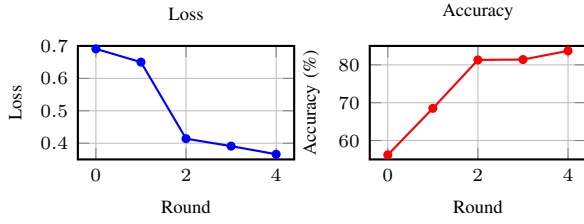


Fig. 7. Async FL validation with $K = 1$.

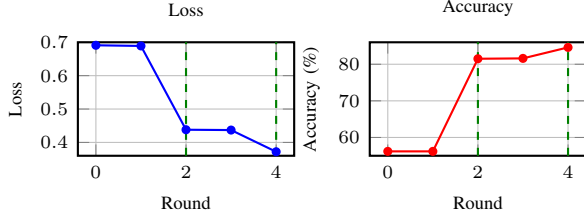


Fig. 8. Async FL with $K = 2$ under IID partitioning.

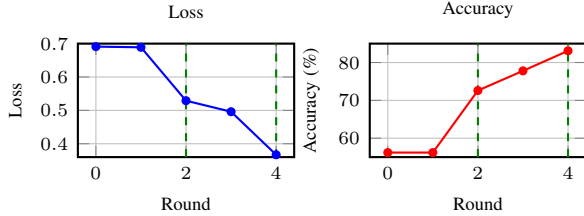


Fig. 9. Async FL with $K = 2$ under non-IID partitioning.

For $K = 2$, the proposed method achieves $83.18 \pm 0.76\%$ accuracy under IID over three seeds and 84.62% under non-IID in a single run, while reducing upload cost by 49.70% and total communication by 24.85% . Fig. 8 and Fig. 9 show that accuracy improves primarily at rounds where full updates are performed, while shallow-only rounds maintain performance with minimal change. We note that this pattern alone does not establish a functional division of labour between shallow and deep layers. The shallow group holds 0.60% of the parameters, so shallow-only rounds necessarily leave the model almost unchanged, and the observation is therefore equally consistent with parameter count as with layer role. Distinguishing the two would require a partition-ratio ablation, which we leave to future work.

For $K = 4$, upload reduction increases to 74.55% , but accuracy decreases to 79.21% under IID and 83.89% under non-IID. Fig. 10 and Fig. 11 show extended plateaus during shallow-only rounds, followed by sharp improvements when deep updates occur. This behavior reflects increased staleness in deep parameters, which slows convergence and reduces final accuracy.

C. Discussion of Results

Upload volume is measured identically for both schedules, as the serialized size of the full `state_dict`. Counting only trainable parameters omits the BatchNorm running statistics, which are transmitted but carry no gradient, understating each

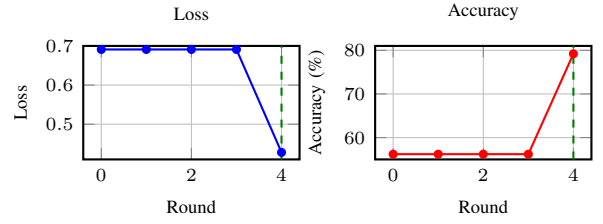


Fig. 10. Async FL with $K = 4$ under IID partitioning.

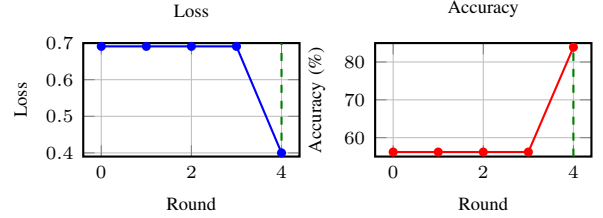


Fig. 11. Async FL with $K = 4$ under non-IID partitioning.

TABLE I
SUMMARY OF EXPERIMENTAL RESULTS. † MEAN $\pm$ STANDARD DEVIATION OVER THREE RANDOM SEEDS; ALL OTHER ROWS ARE SINGLE RUNS. UPLOAD IS MEASURED ON THE FULL `STATE_DICT` FOR BOTH ARMS, AND THE CENTRALIZED ROW IS A NON-FEDERATED REFERENCE RATHER THAN AN UPPER BOUND.

Experiment	Acc. (%)	Upload (MiB)	Red.
Centralized	84.08	—	—
Sync IID †	82.68 ± 0.98	1699.75	0.00%
Sync non-IID	84.53	1699.75	0.00%
Async $K = 1$ IID	83.71	1699.75	0.00%
Async $K = 2$ IID †	83.18 ± 0.76	855.01	49.70%
Async $K = 4$ IID	79.21	432.64	74.55%
Async $K = 2$ non-IID	84.62	855.01	49.70%
Async $K = 4$ non-IID	83.89	432.64	74.55%

model by 3,872 bytes and a four-round, three-client run by 46,464 bytes. That is the entire apparent difference between the synchronous and $K = 1$ configurations; under a single convention the two are identical, as they must be, since $K = 1$ is synchronous FedAvg in communication terms.

Table I summarizes the results. Three key observations emerge. First, accuracy gains are temporally aligned with rounds that include deep parameter updates. Second, $K = 2$ provides the best trade-off, achieving nearly 50% reduction in upload cost while maintaining accuracy close to synchronous FL. Third, accuracy differences between synchronous FedAvg and asynchronous $K = 2$ under IID partitioning are small relative to seed-to-seed variation. Over three seeds the synchronous baseline alone spans 1.91 percentage points (81.85% to 83.76%), which is wider than the mean paired difference of $+0.50$ percentage points between the two schedules ($p = 0.31$). The asynchronous schedule is at least as accurate on every seed tested, but the margin is small and dominated by a single seed. The scheduling change is therefore best characterised as accuracy-neutral at this federation size rather than accuracy-improving, and the communication saving is obtained without a measurable accuracy penalty. The single

non-IID pair (84.62% versus 84.53%) is consistent with this reading but, being one run per configuration, does not by itself establish robustness under heterogeneity.

The $K = 1$ row is algorithmically identical to synchronous FedAvg, yet Table I reports a different accuracy; the two differ only in the floating-point precision used to accumulate the aggregation. From bitwise identical initial parameters on identical partitions they diverge by a maximum relative difference of 1.3×10^{-7} after one round, which is float32 machine epsilon, yet finish 1.86 percentage points apart. With four rounds the model is far from converged and such perturbations are amplified rather than damped. This is the same effect that produces the seed-to-seed spread above, and it is why we treat the accuracy column as indicative and rest our claims on the communication column.

Several limitations bound these results. The federation comprises three clients over four rounds with one local epoch per round, which is small relative to deployed FL systems. The layer partition is fixed a priori and the interval K is constant, with no adaptation to observed layer discrepancy. The shallow group holds only 0.60% of model parameters, so shallow-only rounds carry limited information and the upload saving derives almost entirely from omitting the deep group; this also highlights the need for more balanced partitions or adaptive scheduling in future work. Accuracy is replicated over three seeds only for the IID synchronous and $K = 2$ configurations; the non-IID and $K = 4$ results are single runs and should be read as indicative. Finally, only upload volume was measured. Energy, latency, and congestion benefits are inferred from reduced transmission volume rather than observed, and are stated as potential system-level consequences.

VII. CONCLUSION

In this paper we characterized a fixed-schedule form of layer-wise update scheduling for communication-efficient federated learning on IoT health devices. By partitioning model parameters into shallow and deep groups by parameter mass and synchronizing them at different fixed frequencies, the method reduces uplink communication without architectural modification, without runtime measurement of layer statistics, and without any server-side per-layer state. It therefore applies directly on top of standard FedAvg, which is the property that matters most for severely resource-constrained deployments.

Evaluation on PTB-XL ECG classification demonstrates that $K = 2$ achieves a 49.70% reduction in client upload volume with no accuracy penalty measurable at three seeds, while $K = 4$ increases the reduction to 74.55% at a clear cost in accuracy. The upload reductions are exact consequences of the schedule and the layer partition, and are therefore independent of the seed; the accuracy comparison is not, and is reported with its seed variance.

Several directions remain open. Our evaluation does not include hardware deployment with direct power and latency measurement, a formal convergence analysis under bounded deep-layer staleness, adaptive or learned selection of K and

of the layer grouping, or comparison against gradient compression and split-learning baselines [17]. Extending the evaluation to larger federations, additional architectures, and additional datasets is likewise left to future work.

Overall, the proposed approach provides a simple and practical mechanism for balancing communication efficiency and learning performance in resource-constrained FL deployments.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, and B. A. y Arcas, "Federated learning of deep networks using model averaging," *arXiv preprint arXiv:1602.05629*, 2016.
- [2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, 2019.
- [3] J. Konecny, H. B. McMahan, D. Ramage, and P. Richtarik, "Federated optimization: Distributed machine learning for on-device intelligence," *arXiv preprint arXiv:1610.02527*, 2016.
- [4] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.
- [5] F. Hartmann, "Federated learning," Master's thesis, Freie Universität Berlin, 2018. [Online]. Available: https://www.mi.fu-berlin.de/inf/groups/ag-ti/theses/download/Hartmann_F18.pdf
- [6] Y. Lin, S. Han, H. Mao, Y. Wang, and B. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=SkhQHMW0W>
- [7] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush *et al.*, "Adaptive federated optimization," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=LkFG3IB13U5>
- [8] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: stochastic controlled averaging for federated learning," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, p. 476.
- [9] B. Yuan, S. Ge, and W. Xing, "A federated learning framework for healthcare IoT devices," *arXiv preprint arXiv:2005.05083*, 2020.
- [10] C. Xie, S. Koyejo, and I. Gupta, "Asynchronous federated optimization," *arXiv preprint arXiv:1903.03934*, 2019.
- [11] A. Tariq, M. A. Serhani, F. M. Sallabi, E. S. Barka, T. Qayyum *et al.*, "Trustworthy federated learning: A comprehensive review, architecture, key challenges, and future research prospects," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 4920–4998, 2024.
- [12] S. Lee, T. Zhang, and S. Avestimehr, "Layer-wise adaptive model aggregation for scalable federated learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 7, 2023, pp. 8491–8499.
- [13] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *arXiv preprint arXiv:1812.06127*, 2020.
- [14] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS)*, 2020, p. 638.
- [15] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS)*, vol. 2, 2014, p. 3320–3328.
- [16] P. Wagner, N. Strodthoff, R.-D. Bousseljot, W. Samek, and T. Schaeffter, "PTB-XL, a large publicly available electrocardiography dataset," *Scientific Data*, vol. 7, no. 1, p. 154, 2020.
- [17] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," *arXiv preprint arXiv:1812.00564*, 2018. [Online]. Available: <https://arxiv.org/abs/1812.00564>